

EXERCICE COMPLET — TOUS LES CONCEPTS DU COURS

LaunchPad

L'API d'un centre de contrôle de lancements spatiaux

Service Web (25604) • Projet guidé pas à pas
TypeScript · Express · Prisma · Neon · Axios · JWT

On construit le backend d'un **centre de contrôle de lancements spatiaux**. On récupère les **vrais** prochains décollages de fusées (SpaceX, NASA, ESA, Roscosmos...) depuis une API publique, on les stocke, et chaque utilisateur peut **suivre** ses lancements favoris et **commenter**. Cet exercice mobilise **tous** les concepts du cours. Suivez les étapes dans l'ordre : c'est un projet fil rouge.

Table des matières

1. Objectifs & ce que vous allez construire	2
2. Prérequis	3
Étape 1 : Initialiser le projet TypeScript	3
Étape 2 : Installer les dépendances	3
Étape 3 : Configurer TypeScript et les scripts	4
Étape 4 : Neon + variables d'environnement	4
Étape 5 : Le schéma Prisma (modèles, énums, relations)	4
Étape 6 : Migration + client Prisma (singleton)	6
Étape 7 : Le serveur Express minimal	6
Étape 8 : Sécurité — middlewares JWT	6
Étape 9 : Authentification (register / login / me)	7
Étape 10 : Les lancements (CRUD + filtres + pagination)	8
Étape 11 : Import live via Axios (The Space Devs)	9
Étape 12 : Ma liste de suivi (propriété)	10
Étape 13 : Les commentaires (mission log)	11
Étape 14 : Brancher tous les routers	12
(Optional) Créer un compte ADMIN	13

Étape 15 : Tester le flux complet	13
Checklist finale	14
Dépannage	14
Pour aller plus loin (bonus)	14

1. Objectifs & ce que vous allez construire

À la fin, vous aurez une API avec ces routes :

Méthode	Route	Accès	Concept
POST	/auth/register	public	bcrypt (hachage)
POST	/auth/login	public	JWT (signature)
GET	/auth/me	connecté	JWT (vérification)
GET	/lancements	public	findMany + filtres + pagination
GET	/lancements/:id	public	findUnique
POST	/lancements/importer	connecté	Axios + Prisma
POST	/lancements	ADMIN	RBAC
PATCH	/lancements/:id	ADMIN	RBAC + update
DELETE	/lancements/:id	ADMIN	RBAC + delete
GET	/mes-suivis	connecté	propriété (ownership)
POST	/mes-suivis	connecté	relation N-N
PATCH	/mes-suivis/:id	propriétaire	ownership
DELETE	/mes-suivis/:id	propriétaire	ownership
GET	/lancements/:id/commentaires	public	relation 1-N
POST	/lancements/:id/commentaires	connecté	relation 1-N
DELETE	/commentaires/:id	auteur/ADMIN	ownership + RBAC

Le cœur du système — l'import live (Axios → Prisma) :

```

1 Client --> POST /lancements/importer { "limite": 5 }
2   (1) Express
3   (2) Axios --> The Space Devs API (Launch Library 2)
4       renvoie les 5 prochains lancements REELS
5   (3) transformation (nom, agence, fusee, date, lieu...)
6   (4) Prisma --> Neon (upsert : evite les doublons)
7   --> 201 { importes: 5 }
```

Structure de fichiers visée :

```

1 launchpad/
2   prisma/
3     schema.prisma
4     migrations/
5   generated/prisma/      (client genere, gitignore)
6   src/
7     server.ts
8     api/spaceDevs.ts      (instance Axios)
9     middlewares/auth.ts   (JWT : authentifier + exigerRole)
10    routes/
11      auth.routes.ts
12      lancements.routes.ts
13      suivis.routes.ts
14      commentaires.routes.ts
```

```

15   utils/prisma.ts      (singleton Prisma, adaptateur Neon)
16   .env                 (secrets, gitignore)
17   .gitignore
18   tsconfig.json
19   package.json

```

2. Prérequis

- **Node.js** ≥ 18 (node -v).
- Un compte **Neon** gratuit (PostgreSQL) : <https://neon.tech>.
- **VS Code** + l'extension **REST Client** (ou Postman).
- Des bases en **TypeScript**.

À propos de l'API spatiale. On utilise **The Space Devs — Launch Library 2**, gratuite et **sans clé**. En développement, on tape l'endpoint **11dev** (bac à sable) qui a une limite de requêtes plus généreuse : <https://11dev.thespacedevs.com/2.2.0>. (En production : <https://11.thespacedevs.com/2.2.0>, limité à environ 15 requêtes/heure en anonyme.)

Étape 1 : Initialiser le projet TypeScript

```

1 mkdir launchpad
2 cd launchpad
3 npm init -y

```

Créez un `.gitignore` tout de suite :

```

1 node_modules/
2 .env
3 generated/
4 dist/

```

Vérification : `package.json` et `.gitignore` existent.

Étape 2 : Installer les dépendances

```

1 # Production
2 npm install express dotenv @prisma/client @prisma/adapters-neon axios bcryptjs jsonwebtoken
3
4 # Développement
5 npm install --save-dev typescript tsx prisma @types/node @types/express @types/bcryptjs @types/jsonwebtoken

```

Paquet	Rôle
express	l'API REST
@prisma/client + prisma	l'ORM (requêtes + migrations)
@prisma/adapters-neon	connecter Prisma à Neon
axios	consommer l'API The Space Devs
bcryptjs	hacher les mots de passe (sans compilation native)
jsonwebtoken	signer / vérifier les JWT
dotenv	charger le <code>.env</code>
tsx	exécuter le TypeScript (dev)

Étape 3 : Configurer TypeScript et les scripts

```
1 npx tsc --init
```

Dans `tsconfig.json`, assurez-vous d'avoir au moins :

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "module": "NodeNext",
5     "moduleResolution": "NodeNext",
6     "strict": true,
7     "esModuleInterop": true,
8     "skipLibCheck": true,
9     "outDir": "dist",
10    "rootDir": "src"
11  }
12 }
```

Ajoutez les **scripts** dans `package.json` :

```
1 "scripts": {
2   "dev": "tsx watch src/server.ts"
3 }
```

Étape 4 : Neon + variables d'environnement

Sur <https://neon.tech>, créez un projet et copiez la **chaîne de connexion**. À la racine, créez `.env` :

```
1 DATABASE_URL="postgresql://utilisateur:motdepasse@hote.neon.tech/nomdb?sslmode=require"
2 JWT_SECRET="changez_moi_par_une_longue_chaine_aleatoire"
3 SPACE_API="https://1ldev.thespacedevs.com/2.2.0"
4 PORT=3000
```

Générez un bon secret JWT :

```
1 node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Initialisez Prisma : `npx prisma init`

Le `.env` est dans `.gitignore`. Ne le committez **jamais**.

Étape 5 : Le schéma Prisma (modèles, énums, relations)

Remplacez tout `prisma/schema.prisma` par :

```
1 generator client {
2   provider = "prisma-client"
3   output   = "../generated/prisma"
4 }
5
6 datasource db {
7   provider = "postgresql"
8   url      = env("DATABASE_URL")
9 }
10
11 // ----- ENUMS -----
12 enum Role {
13   USER
14   ADMIN
```

```

15 }
16
17 enum Statut {
18     A_VENIR
19     SUCCES
20     ECHEC
21     REPORTE
22 }
23
24 // ----- MODELES -----
25 model User {
26     id          Int          @id @default(autoincrement())
27     email       String       @unique
28     pseudo      String
29     password    String       // HASH bcrypt
30     role        Role         @default(USER)
31     createdAt   DateTime     @default(now())
32     suivis      Suivi[]
33     commentaires Commentaire[]
34 }
35
36 model Lancement {
37     id          Int          @id @default(autoincrement())
38     ref         String       @unique           // id externe -> anti-doublon
39     nom         String
40     agence      String?
41     fusee       String?
42     mission     String?
43     lieu        String?
44     imageUrl    String?
45     dateLancement DateTime?
46     statut      Statut       @default(A_VENIR)
47     createdAt   DateTime     @default(now())
48     suivis      Suivi[]
49     commentaires Commentaire[]
50 }
51
52 // Liste de suivi : un User <-> un Lancement (N-N enrichie)
53 model Suivi {
54     id          Int          @id @default(autoincrement())
55     rappel      Boolean      @default(false)
56     userId      Int
57     lancementId Int
58     user        User         @relation(fields: [userId], references: [id])
59     lancement   Lancement    @relation(fields: [lancementId], references: [id])
60
61     @@unique([userId, lancementId])
62 }
63
64 model Commentaire {
65     id          Int          @id @default(autoincrement())
66     contenu     String
67     createdAt   DateTime     @default(now())
68     userId      Int
69     lancementId Int
70     user        User         @relation(fields: [userId], references: [id])
71     lancement   Lancement    @relation(fields: [lancementId], references: [id])
72 }

```

À comprendre. 2 énums (Role, Statut). Lancement.ref est @unique : c'est l'identifiant externe de l'API, il empêche d'importer deux fois le même lancement. Suivi relie User et Lancement (relation N-N enrichie d'un booléen rappel), avec @@unique([userId,

lancementId])). Commentaire est une relation 1-N.

Étape 6 : Migration + client Prisma (singleton)

```
1 npx prisma migrate dev --name init
2 npx prisma generate
```

Créez le **singleton** dans `src/utils/prisma.ts` :

```
1 import { PrismaNeon } from "@prisma/adapter-neon"
2 import { PrismaClient } from "../../generated/prisma/client"
3 import dotenv from "dotenv"
4 dotenv.config()
5
6 const adapter = new PrismaNeon({ connectionString: process.env.DATABASE_URL })
7
8 const prisma = new PrismaClient({
9   adapter,
10  log: ["query", "info", "warn", "error"],
11 })
12
13 export default prisma
```

L'import vient de `../../generated/prisma/client` (dossier output du schéma), **pas** de `@prisma/client`.

Étape 7 : Le serveur Express minimal

Créez `src/server.ts` :

```
1 import express, { Request, Response } from "express"
2 import dotenv from "dotenv"
3 dotenv.config()
4
5 const app = express()
6 app.use(express.json())
7
8 app.get("/", (req: Request, res: Response) => {
9   res.json({ message: "LaunchPad - Mission Control" })
10 })
11
12 const PORT = process.env.PORT || 3000
13 app.listen(PORT, () => console.log(`Serveur sur http://localhost:${PORT}`))
```

```
1 npm run dev
```

Vérification : le terminal affiche `Serveur sur http://localhost:3000`.

Étape 8 : Sécurité — middlewares JWT

Créez `src/middlewares/auth.ts` :

```
1 import { Request, Response, NextFunction } from "express"
2 import jwt from "jsonwebtoken"
3
4 export type JwtPayload = { sub: number; role: "USER" | "ADMIN" }
5
```

```

6 // Verifie le token et attache l'utilisateur a req.user
7 export function authentifier(req: Request, res: Response, next: NextFunction) {
8   const header = req.headers.authorization // "Bearer xxx.yyy.zzz"
9   if (!header?.startsWith("Bearer ")) {
10     return res.status(401).json({ erreur: "Token manquant" })
11   }
12   const token = header.split(" ")[1]
13   try {
14     const payload = jwt.verify(token, process.env.JWT_SECRET!) as JwtPayload
15     ;(req as any).user = payload
16     next()
17   } catch {
18     res.status(401).json({ erreur: "Token invalide ou expire" })
19   }
20 }
21
22 // Exige un role (a brancher APRES authentifier)
23 export function exigerRole(role: "ADMIN" | "USER") {
24   return (req: Request, res: Response, next: NextFunction) => {
25     if ((req as any).user?.role !== role) {
26       return res.status(403).json({ erreur: "Acces refuse" })
27     }
28     next()
29   }
30 }

```

401 vs 403 : 401 = « je ne sais pas qui tu es » ; 403 = « je sais, mais tu n'as pas le droit ».

Étape 9 : Authentification (register / login / me)

Créez src/routes/auth.routes.ts :

```

1 import { Router, Request, Response } from "express"
2 import bcrypt from "bcryptjs"
3 import jwt from "jsonwebtoken"
4 import prisma from "../utils/prisma"
5 import { authentifier } from "../middlewares/auth"
6
7 const router = Router()
8
9 // POST /auth/register
10 router.post("/register", async (req: Request, res: Response) => {
11   const { email, pseudo, password } = req.body
12   if (!email || !pseudo || !password) {
13     return res.status(400).json({ erreur: "email, pseudo et password requis" })
14   }
15   try {
16     const hash = await bcrypt.hash(password, 10)
17     const user = await prisma.user.create({ data: { email, pseudo, password: hash } })
18     res.status(201).json({ id: user.id, email: user.email, pseudo: user.pseudo })
19   } catch {
20     res.status(400).json({ erreur: "Email deja utilise" })
21   }
22 })
23
24 // POST /auth/login
25 router.post("/login", async (req: Request, res: Response) => {
26   const { email, password } = req.body
27   const user = await prisma.user.findUnique({ where: { email } })
28   if (!user) return res.status(401).json({ erreur: "Identifiants invalides" })
29   const ok = await bcrypt.compare(password, user.password)
30   if (!ok) return res.status(401).json({ erreur: "Identifiants invalides" })

```

```

31
32   const token = jwt.sign(
33     { sub: user.id, role: user.role },
34     process.env.JWT_SECRET!,
35     { expiresIn: "2h" }
36   )
37   res.json({ token })
38 })
39
40 // GET /auth/me (route protegee)
41 router.get("/me", authentifier, async (req: Request, res: Response) => {
42   const id = (req as any).user.sub
43   const user = await prisma.user.findUnique({
44     where: { id },
45     select: { id: true, email: true, pseudo: true, role: true, createdAt: true },
46   })
47   res.json(user)
48 })
49
50 export default router

```

Sécurité : on utilise `select` pour **ne jamais** renvoyer `password`. Le message d'erreur de connexion reste générique (on ne révèle pas si c'est l'email ou le mot de passe qui est faux).

Étape 10 : Les lancements (CRUD + filtres + pagination)

Créez `src/routes/lancements.routes.ts` :

```

1 import { Router, Request, Response } from "express"
2 import prisma from "../utils/prisma"
3 import { authentifier, exigerRole } from "../middlewares/auth"
4
5 const router = Router()
6
7 // GET /lancements?statut=A_VENIR&page=1&limit=10 (public, filtre + pagine)
8 router.get("/", async (req: Request, res: Response) => {
9   const { statut } = req.query
10   const page = Math.max(1, Number(req.query.page) || 1)
11   const limit = Math.min(50, Number(req.query.limit) || 10)
12
13   const where = statut ? { statut: String(statut) as any } : {}
14   const [lancements, total] = await Promise.all([
15     prisma.lancement.findMany({
16       where, skip: (page - 1) * limit, take: limit,
17       orderBy: { dateLancement: "asc" },
18     }),
19     prisma.lancement.count({ where }),
20   ])
21   res.json({ page, limit, total, lancements })
22 })
23
24 // GET /lancements/:id (public)
25 router.get("/:id", async (req: Request, res: Response) => {
26   const id = Number(req.params.id)
27   const lancement = await prisma.lancement.findUnique({ where: { id } })
28   if (!lancement) return res.status(404).json({ erreur: "Lancement introuvable" })
29   res.json(lancement)
30 })
31
32 // POST /lancements (ADMIN : creation manuelle)
33 router.post("/", authentifier, exigerRole("ADMIN"), async (req: Request, res: Response) => {
34   const { ref, nom, agence, fusee } = req.body

```

```

35   if (!ref || !nom) return res.status(400).json({ erreur: "ref et nom requis" })
36   const lancement = await prisma.lancement.create({ data: { ref, nom, agence, fusee } })
37   res.status(201).json(lancement)
38 })
39
40 // PATCH /lancements/:id (ADMIN)
41 router.patch("/:id", authentifier, exigerRole("ADMIN"), async (req: Request, res: Response) => {
42   const id = Number(req.params.id)
43   try {
44     const lancement = await prisma.lancement.update({ where: { id }, data: req.body })
45     res.json(lancement)
46   } catch {
47     res.status(404).json({ erreur: "Lancement introuvable" })
48   }
49 })
50
51 // DELETE /lancements/:id (ADMIN)
52 router.delete("/:id", authentifier, exigerRole("ADMIN"), async (req: Request, res: Response) => {
53   const id = Number(req.params.id)
54   try {
55     await prisma.lancement.delete({ where: { id } })
56     res.status(204).end()
57   } catch {
58     res.status(404).json({ erreur: "Lancement introuvable" })
59   }
60 })
61
62 export default router

```

Concepts : filtres query string, pagination (skip/take + count), RBAC (ADMIN).

Étape 11 : Import live via Axios (The Space Devs)

L'instance Axios dans src/api/spaceDevs.ts :

```

1 import axios from "axios"
2 import dotenv from "dotenv"
3 dotenv.config()
4
5 export const spaceDevs = axios.create({
6   baseURL: process.env.SPACE_API || "https://1ldev.thespacedevs.com/2.2.0",
7   timeout: 10000,
8 })

```

Ajoutez la route d'import dans **lancements.routes.ts** (avant export default router) :

```

1 import axios from "axios"
2 import { spaceDevs } from "../api/spaceDevs"
3
4 // POST /lancements/importer  body: { "limite": 5 }
5 // -> recupere les prochains lancements REELS et les enregistre (sans doublon)
6 router.post("/importer", authentifier, async (req: Request, res: Response) => {
7   const limite = Math.min(10, Number(req.body.limite) || 5)
8   try {
9     // 1) Appel API publique via Axios
10    const { data } = await spaceDevs.get("/launch/upcoming/", { params: { limit: limite } })
11
12    // 2) Transformation + 3) ecriture (upsert : creer si nouveau, sinon mettre a jour)
13    let importes = 0
14    for (const l of data.results) {
15      await prisma.lancement.upsert({
16        where: { ref: l.id },
17        update: { statut: "A_VENIR" },

```

```

18     create: {
19         ref: l.id,
20         nom: l.name,
21         agence: l.launch_service_provider?.name ?? null,
22         fusee: l.rocket?.configuration?.name ?? null,
23         mission: l.mission?.name ?? null,
24         lieu: l.pad?.location?.name ?? null,
25         imageUrl: l.image ?? null,
26         dateLancement: l.net ? new Date(l.net) : null,
27         statut: "A_VENIR",
28     },
29 })
30 importes++
31 }
32 res.status(201).json({ message: `${importes} lancement(s) importe(s)`, importes })
33 } catch (e) {
34     if (axios.isAxiosError(e)) {
35         return res.status(502).json({ erreur: "API The Space Devs injoignable (ou quota atteint)" })
36     }
37     res.status(500).json({ erreur: "Erreur lors de l'import" })
38 }
39 })

```

Ordre des routes ! POST /importer doit être déclaré **avant** une éventuelle route POST /:id.
Règle générale : routes **littérales** avant routes **paramétrées**.

Concepts : instance Axios (baseUrl, timeout), params, **transformation** de la réponse (?, ??), axios.isAxiosError, et **upsert** Prisma qui évite les doublons grâce au champ ref @unique.

Étape 12 : Ma liste de suivi (propriété)

Créez src/routes/suivis.routes.ts :

```

1 import { Router, Request, Response } from "express"
2 import prisma from "../utils/prisma"
3 import { authentifier } from "../middlewares/auth"
4
5 const router = Router()
6 router.use(authentifier) // TOUT ce router exige d'être connecté
7
8 // GET /mes-suivis -> les lancements suivis par l'utilisateur connecté
9 router.get("/", async (req: Request, res: Response) => {
10     const userId = (req as any).user.sub
11     const suivis = await prisma.suivi.findMany({
12         where: { userId }, include: { lancement: true }, orderBy: { id: "desc" },
13     })
14     res.json(suivis)
15 })
16
17 // POST /mes-suivis body: { "lancementId": 1, "rappel": true }
18 router.post("/", async (req: Request, res: Response) => {
19     const userId = (req as any).user.sub
20     const { lancementId, rappel } = req.body
21     try {
22         const suivi = await prisma.suivi.create({
23             data: { userId, lancementId: Number(lancementId), rappel: !!rappel },
24         })
25         res.status(201).json(suivi)
26     } catch {
27         res.status(400).json({ erreur: "Lancement déjà suivi (ou inexistant)" })
28     }
29 })

```

```

28   }
29   })
30
31   // PATCH /mes-suivis/:id -> activer/desactiver le rappel (propriete verifiee)
32   router.patch("/:id", async (req: Request, res: Response) => {
33     const userId = (req as any).user.sub
34     const id = Number(req.params.id)
35     const suivi = await prisma.suivi.findUnique({ where: { id } })
36     if (!suivi) return res.status(404).json({ erreur: "Suivi introuvable" })
37     if (suivi.userId !== userId) {
38       return res.status(403).json({ erreur: "Ce n'est pas votre suivi" })
39     }
40     const maj = await prisma.suivi.update({ where: { id }, data: { rappel: !!req.body.rappel } })
41     res.json(maj)
42   })
43
44   // DELETE /mes-suivis/:id (propriete verifiee)
45   router.delete("/:id", async (req: Request, res: Response) => {
46     const userId = (req as any).user.sub
47     const id = Number(req.params.id)
48     const suivi = await prisma.suivi.findUnique({ where: { id } })
49     if (!suivi) return res.status(404).json({ erreur: "Suivi introuvable" })
50     if (suivi.userId !== userId) {
51       return res.status(403).json({ erreur: "Ce n'est pas votre suivi" })
52     }
53     await prisma.suivi.delete({ where: { id } })
54     res.status(204).end()
55   })
56
57   export default router

```

Concept clé : la propriété (ownership). Être connecté ne suffit pas : on vérifie que le suivi **appartient** à l'utilisateur (`suivi.userId === userId`), sinon 403. `router.use(authentifier)` protège **toutes** les routes du router d'un coup.

Étape 13 : Les commentaires (mission log)

Créez `src/routes/commentaires.routes.ts` :

```

1  import { Router, Request, Response } from "express"
2  import prisma from "../utils/prisma"
3  import { authentifier } from "../middlewares/auth"
4
5  const router = Router()
6
7  // GET /lancements/:id/commentaires (public)
8  router.get("/lancements/:id/commentaires", async (req: Request, res: Response) => {
9    const lancementId = Number(req.params.id)
10    const commentaires = await prisma.commentaire.findMany({
11      where: { lancementId },
12      include: { user: { select: { pseudo: true } } },
13      orderBy: { createdAt: "desc" },
14    })
15    res.json(commentaires)
16  })
17
18  // POST /lancements/:id/commentaires body: { "contenu": "..." } (connecte)
19  router.post("/lancements/:id/commentaires", authentifier, async (req: Request, res: Response) => {
20    const userId = (req as any).user.sub
21    const lancementId = Number(req.params.id)
22    const { contenu } = req.body
23    if (!contenu) return res.status(400).json({ erreur: "contenu requis" })

```

```

24  const commentaire = await prisma.commentaire.create({ data: { contenu, userId, lancementId } })
25  res.status(201).json(commentaire)
26  })
27
28  // DELETE /commentaires/:id -> l'auteur OU un ADMIN peut supprimer
29  router.delete("/commentaires/:id", authentifier, async (req: Request, res: Response) => {
30    const user = (req as any).user // { sub, role }
31    const id = Number(req.params.id)
32    const commentaire = await prisma.commentaire.findUnique({ where: { id } })
33    if (!commentaire) return res.status(404).json({ erreur: "Commentaire introuvable" })
34
35    const estAuteur = commentaire.userId === user.sub
36    const estAdmin = user.role === "ADMIN"
37    if (!estAuteur && !estAdmin) {
38      return res.status(403).json({ erreur: "Action non autorisee" })
39    }
40    await prisma.commentaire.delete({ where: { id } })
41    res.status(204).end()
42  })
43
44  export default router

```

Concepts : relation 1-N, include + select (joindre le pseudo), et autorisation **mixte** (auteur ou ADMIN).

Étape 14 : Brancher tous les routers

Mettez à jour `src/server.ts` :

```

1  import express, { Request, Response, NextFunction } from "express"
2  import dotenv from "dotenv"
3  import authRoutes from "./routes/auth.routes"
4  import lancementsRoutes from "./routes/lancements.routes"
5  import suivisRoutes from "./routes/suivis.routes"
6  import commentairesRoutes from "./routes/commentaires.routes"
7
8  dotenv.config()
9
10 const app = express()
11 app.use(express.json())
12
13 app.get("/", (req: Request, res: Response) => res.json({ message: "LaunchPad" }))
14
15 app.use("/auth", authRoutes)
16 app.use("/lancements", lancementsRoutes)
17 app.use("/mes-suivis", suivisRoutes)
18 app.use("/", commentairesRoutes) // /lancements/:id/commentaires et /commentaires/:id
19
20 // 404
21 app.use((req: Request, res: Response) => res.status(404).json({ erreur: "Route inconnue" }))
22
23 // gestionnaire d'erreurs global (4 parametres)
24 app.use((err: any, req: Request, res: Response, next: NextFunction) => {
25   console.error(err)
26   res.status(500).json({ erreur: "Erreur interne du serveur" })
27 })
28
29 const PORT = process.env.PORT || 3000
30 app.listen(PORT, () => console.log(`Serveur sur http://localhost:${PORT}`))

```

(Optionnel) Créer un compte ADMIN

Le `register` crée toujours un `USER`. Pour tester les routes `ADMIN`, promouvez un compte via **Prisma Studio** : `npx prisma studio` → `table User` → `role = ADMIN` → `Save`.

Étape 15 : Tester le flux complet

Créez `test.rest` à la racine (extension **REST Client**) :

```
1 @base = http://localhost:3000
2
3 ### 1. Inscription
4 POST {{base}}/auth/register
5 Content-Type: application/json
6
7 { "email": "neil@cmaisonneuve.qc.ca", "pseudo": "Neil", "password": "apollo11" }
8
9 ### 2. Connexion -> copiez le token renvoye
10 # @name login
11 POST {{base}}/auth/login
12 Content-Type: application/json
13
14 { "email": "neil@cmaisonneuve.qc.ca", "password": "apollo11" }
15
16 ### 3. Qui suis-je ? (route protegee)
17 GET {{base}}/auth/me
18 Authorization: Bearer {{login.response.body.token}}
19
20 ### 4. Importer les vrais prochains lancements (Axios)
21 POST {{base}}/lancements/importer
22 Authorization: Bearer {{login.response.body.token}}
23 Content-Type: application/json
24
25 { "limite": 5 }
26
27 ### 5. Lister les lancements (public, filtre + pagine)
28 GET {{base}}/lancements?statut=A_VENIR&page=1&limit=10
29
30 ### 6. Suivre le lancement 1 (avec rappel)
31 POST {{base}}/mes-suivis
32 Authorization: Bearer {{login.response.body.token}}
33 Content-Type: application/json
34
35 { "lancementId": 1, "rappel": true }
36
37 ### 7. Voir ma liste de suivi
38 GET {{base}}/mes-suivis
39 Authorization: Bearer {{login.response.body.token}}
40
41 ### 8. Commenter le lancement 1
42 POST {{base}}/lancements/1/commentaires
43 Authorization: Bearer {{login.response.body.token}}
44 Content-Type: application/json
45
46 { "contenu": "Decollage imminent, j'ai hate !" }
47
48 ### 9. Voir les commentaires (public)
49 GET {{base}}/lancements/1/commentaires
50
51 ### 10. Importer SANS token -> doit echouer (401)
52 POST {{base}}/lancements/importer
53 Content-Type: application/json
54
55 { "limite": 3 }
```

```

56
57 ### 11. Supprimer le lancement 1 en tant que USER -> doit echouer (403)
58 DELETE {{base}}/lancements/1
59 Authorization: Bearer {{login.response.body.token}}

```

Résultats attendus :

Test	Code	Pourquoi
Register	201	compte créé (sans le hash)
Login	200	renvoie un token
/auth/me avec token	200	profil (sans password)
Import avec token	201	Axios + upsert réussis
Import sans token	401	non authentifié
Suivre un lancement	201	relation créée
Commenter	201	relation 1-N
DELETE lancement en USER	403	pas le rôle ADMIN

Checklist finale

- ☐ **TypeScript** : projet sous `tsx watch`, types stricts.
- ☐ **Prisma + Neon** : schéma migré, ≥ 4 modèles, 2 énums, relations (1-N et N-N).
- ☐ **Express** : routers séparés, CRUD complet, 404 + handler d'erreurs.
- ☐ **Axios** : import live (instance + `params` + transformation + `isAxiosError`).
- ☐ **JWT** : register (hash bcrypt), login (token + expiration), `/me` protégé.
- ☐ **Autorisation** : routes ADMIN (RBAC) et propriété (ownership) ; 401 vs 403.
- ☐ **Sécurité** : password jamais renvoyé, `JWT_SECRET` en `.env`, `.env` gitignoré.

Dépannage

Problème	Cause	Solution
Cannot find module '../..../generated/...'	client non généré	<code>npx prisma generate</code>
Can't reach database server table "User" does not exist	base Neon en pause migration non appliquée	ouvrir le dashboard Neon <code>npx prisma migrate dev</code>
Import renvoie 502 après plusieurs essais	quota API atteint	utiliser <code>lldev</code> , espacer les ap- pels
<code>req.body</code> est undefined	<code>express.json()</code> mal placé	le mettre avant les routes
Token manquant	header mal formé	<code>Authorization: Bearer <token></code>
Erreur sur <code>user.role</code>	<code>exigerRole</code> avant authentifier	mettre <code>authentifier</code> en premier

Pour aller plus loin (bonus)

- **Validation** des entrées avec **Zod**.
- **Compte à rebours** : une route qui calcule le temps restant avant chaque décollage suivi.
- **Recherche** par agence (`where: { agence: { contains: q } }`).
- **Rafraîchissement** : une route ADMIN qui ré-importe et met à jour les statuts (`upsert`).
- **Statistiques** : nombre de lancements par agence (`groupBy`).
- **Refresh token** + route `/auth/logout`.
- **Déploiement** sur Render / Railway (`npx prisma migrate deploy`).

Décollage réussi ! Vous avez bâti une API complète, typée, persistée et sécurisée, branchée sur de *vraies données spatiales en direct*.